

1. Imię i nazwisko: Zaręba Dominik

2. Przebieg budowania pakietów RPM metodą alternatywną

1) Przygotowanie środowiska do pracy:

```
docker run -it --name opensuse_rpm opensuse/leap:15.6 /bin/bash
```

Na początku przygotowujemy sobie środowisko do pracy. W tym przypadku wybrałem izolowany kontener Docker z systemem openSUSE Leap 15.6, dzięki temu będziemy mieli zapewnione czyste środowisko, aby uniknąć konfliktów z bibliotekami hosta.

2) Aktualizacje systemu/Instalacja narzędzi programistycznych:

a) zypper ref - odświeżenie pamięci podręcznej repozytoriów:

przed:

```
ac18f033d3e2:/ # zypper ref
Looking for gpg keys in repository Update repository of openSUSE Backports.
  gpgkey=http://download.opensuse.org/update/leap/15.6/backports/repodata/repomd.xml.key
Retrieving repository 'Update repository of openSUSE Backports' metadata .....
.....[done]
Building repository 'Update repository of openSUSE Backports' cache .....
.....[done]
Looking for gpg keys in repository Update repository with updates from SUSE Linux Enterprise
```

po:

```
Warning: The gpg key signing file 'repomd.xml' has expired.
Repository:      Main Repository
Key Fingerprint: AD48 5664 E901 B867 051A B15F 35A2 F86E 29B7 00A4
Key Name:        openSUSE Project Signing Key <opensuse@opensuse.org>
Key Algorithm:   RSA 4096
Key Created:     Mon Jun 20 14:03:14 2022
Key Expires:     Fri Jun 19 14:03:14 2026 (EXPIRED)
Rpm Name:        gpg-pubkey-29b700a4-62b07e22
Retrieving repository 'Main Repository' metadata .....
.....[done]
Building repository 'Main Repository' cache .....
.....[done]
All repositories have been refreshed.
```

b) zypper up - aktualizacja wszystkich zainstalowanych pakietów do najnowszych wersji, dostępnych w naszych skonfigurowanych repozytoriach:

przed:

```
ac18f033d3e2:/ # zypper up
Loading repository data...
Warning: Repository 'Main Update Repository' metadata expired since 2026-06-17 10:34:19 UTC.

Warning: Repository metadata expired: Check if 'autorefresh' is turned on (zypper lr), otherwise
```

po:

```
you are using a broken mirror or the server has actually discontinued to support the repository.  
Reading installed packages...  
Nothing to do.
```

c) Zainstalowanie potrzebnych pakietów:

przed:

```
ac18f033d3e2:/ # zypper install -y rpm-build rpmdevtools gcc gcc-c++ make openssl-devel wget  
Loading repository data...  
Warning: Repository 'Main Update Repository' metadata expired since 2026-06-17 10:34:19 UTC.  
Warning: Repository metadata expired: Check if 'autorefresh' is turned on (zypper lr), other
```

po:

```
[done]  
(65/68) Installing: gcc-7-3.9.1.x86_64 .....  
[done]  
(66/68) Installing: libopenssl-devel-3.1.4-150600.2.1.noarch .....  
[done]  
(67/68) Installing: gcc-c++-7-3.9.1.x86_64 .....  
[done]  
(68/68) Installing: rpm-build-4.14.3-150400.59.16.1.x86_64 .....  
[done]
```

Instalowane pakiety to między innymi:

- rpm-build, który dostarcza polecenie rpmbuild potrafiące przeczytać plik .spec, generując na jego podstawie plik .rpm
- rpmdevtools - zestaw narzędzi pomocniczych dla deweloperów pakietów RPM. Będzie potrzebny między innymi do komendy rpmdev-setuptree
- gcc, gcc-c++ - kompilatory C - podstawa, bo aria2, z której będziemy korzystać jest napisana w języku C++
- make - Automatyzator procesu kompilacji - czyta pliki Makefile i zarządza jak i za pomocą jakich poleceń kompilować poszczególne pliki źródłowe.

Lista pakietów, którą wykorzystywaliśmy na zajęciach była znacznie dłuższa i różni się od tej powyższej:

Główną różnicą jest brak zależności od języka Ruby - Narzędzie fpm, z którego tam korzystaliśmy zostało napisane w języku programowania Ruby. Aby w ogóle je uruchomić, musieliśmy najpierw zainstalować środowisko Rubiego, czyli w tym przypadku ruby-devel, a następnie użyć jego własnego menedżera pakietów - gem install, aby pociągnąć fpm i jego wtyczki (fii, czy io-like). W naszej alternatywnej metodzie używamy rpmbuild, które jest natywnym, wbudowanym narzędziem systemów Linux opartych na RPM, dzięki temu omijamy tzw. budowanie środowiska w środowisku.

Na zajęciach dodatkowo tworzyliśmy ręcznie strukturę plików, a pliki instalacyjne trafiały do przypadkowego folderu /tmp - w alternatywnej metodzie wykorzystujemy rpmdevtools i jego rpmdev-setuptree, która łatwo i przyjemnie generuje prawidłową strukturę katalogów - BUILD, SOURCES, SPECS etc.

Na zajęciach dodatkowo instalowaliśmy także takie pakiety jak automake, autoconf, czy libtool. Te narzędzia służą do generowania skryptów ./configure. W przypadku aria2 pobranej w wersji 1.37.0, skrypt ./configure jest już gotowy i dołączony do archiwum - gdybyśmy pobrali surowy kod z repo, komendą git clone - wtedy potrzebowalibyśmy wskazanych pakietów. W tym przypadku udało nam się sprawić, że środowisko jest lżejsze i szybciej się buduje, ale należy mieć na uwadze, że czasami pakiety mogą być wymagane.

3) Użycie polecenia rpmddev-setuptree:

przed:

```
.....[done]  
ac18f033d3e2:/ # rpmddev-setuptree
```

po:

```
ac18f033d3e2:~/rpmbuild # ls -l ~/rpmbuild/  
total 24  
drwxr-xr-x 3 root root 4096 Jun 20 15:25 BUILD  
drwxr-xr-x 2 root root 4096 Jun 20 15:28 BUILDROOT  
drwxr-xr-x 3 root root 4096 Jun 20 15:27 RPMS  
drwxr-xr-x 2 root root 4096 Jun 20 15:16 SOURCES  
drwxr-xr-x 2 root root 4096 Jun 20 15:18 SPECS  
drwxr-xr-x 2 root root 4096 Jun 20 15:16 SRPMS  
ac18f033d3e2:~/rpmbuild #
```

4) Pobranie kodu źródłowego do katalogu RESOURCES:

przed:

```
ac18f033d3e2:/ # cd ~/rpmbuild/SOURCES/  
ac18f033d3e2:~/rpmbuild/SOURCES # wget https://github.com/aria2/aria2/releases/download/release-1.37.0/aria2-1.37.0.tar.gz  
--2026-06-20 15:16:54-- https://github.com/aria2/aria2/releases/download/release-1.37.0/aria2-1.37.0.tar.gz  
Resolving github.com (github.com)... 140.82.121.3  
Connecting to github.com (github.com)[140.82.121.3]:443... connected.  
HTTP request sent, awaiting response... 302 Found  
https://github.com/aria2/aria2/releases/download/release-1.37.0/aria2-1.37.0.tar.gz
```

po:

```
aria2-1.37.0.tar.gz          100%[=====>]      3.72M  
15.8MB/s   in 0.2s  
2026-06-20 15:16:54 (15.8 MB/s) - 'aria2-1.37.0.tar.gz' saved [3904613/3904613]
```

5) Przygotowanie pliku .spec:

```
Name:          aria2c
Version:       1.37.0
Release:       1
Summary:       High speed download utility
License:       GPL
Source0:       aria2-1.37.0.tar.gz

%define _unpackaged_files_terminate_build 0

%description
Aria2 RPM package built using rpmbuild.
%prep
%setup -q -n aria2-1.37.0

%build
./configure --prefix=/usr
make %{?_smp_mflags}

%install
rm -rf %{buildroot}
make install DESTDIR=%{buildroot}

%files
%{_bindir}/aria2c
%{_mandir}/man1/aria2c.1*
%{_mandir}/*/man1/aria2c.1*
%{_datadir}/doc/aria2/
%{_datadir}/locale/*/LC_MESSAGES/aria2.mo

%clean
rm -rf %{buildroot}
```

Plik .spec to plik konfiguracyjny. Łączy w sobie definicję metadanych, wywołania makr preprocesora RPM oraz zbiór skryptów powłoki.

Trochę doinformowałem się na temat tej metody i dowiedziałem się, że jest to standard produkcyjny - plik .spec przechowuje się w systemie kontroli wersji, a kiedy deweloper wypuszcza nową wersję - np. aria2, potok automatyzacji pobiera nowy kod z repo, wstrzykuje do czystego kontenera wraz z plikiem .spec i generuje paczki binarne bez jakiegokolwiek ingerencji człowieka, gwarantując taki sam wynik każdej kompilacji na podstawie tego samego wejścia - podejście deterministyczne. W przypadku fpm, nie mamy możliwości zbudowania takiej automatyzacji.

- %prep i %setup - automatyczne rozpakowanie archiwum i wejście do odpowiedniego katalogu.
- %build - definicja struktury kompilacji ./configure i make
- %install - instalacja plików do specjalnego wirtualnego środowiska
- %files - bezpośrednie wskazanie, które pliki binarne i dokumentacyjne mają ostatecznie trafić do paczki.

6) Budowa pakietu: Uruchomienie komendy `rpmbuild -bb aria2c.spec`, która automatycznie zrealizowała wszystkie kroki opisane w pliku konfiguracyjnym, generując gotowy pakiet `.rpm` w katalogu `RPMS`.

przed:

```
ac18f033d3e2:~/rpmbuild/SPECS #
ac18f033d3e2:~/rpmbuild/SPECS # rpmbuild -bb aria2c.spec
Executing(%prep): /bin/sh -e /var/tmp/rpm-tmp.oVTdin
+ umask 022
+ cd /root/rpmbuild/BUILD
+ cd /root/rpmbuild/BUILD
+ rm -rf aria2-1.37.0
+ /usr/bin/gzip -dc /root/rpmbuild/SOURCES/aria2-1.37.0.tar.gz
+ /usr/bin/tar -xof-
```

po:

```
Processing files: aria2c-1.37.0-1.x86_64
Provides: aria2c = 1.37.0-1 aria2c(x86-64) = 1.37.0-1
Requires(rpmlib): rpmlib(CompressedFileNames) <= 3.0.4-1 rpmlib(FileDigests) <= 4.6.0-1 rpmlib(PayloadFilesHavePrefix) <= 4.0-1
Requires: libc.so.6()(64bit) libc.so.6(GLIBC_2.10)(64bit) libc.so.6(GLIBC_2.14)(64bit) libc.so.6(GLIBC_2.2.5)(64bit) libc.so.6(GLIBC_2.3)(64bit) libc.so.6(GLIBC_2.3.2)(64bit) libc.so.6(GLIBC_2.33)(64bit) libc.so.6(GLIBC_2.34)(64bit) libc.so.6(GLIBC_2.38)(64bit) libcrypto.so.3()(64bit) libcrypto.so.3(OPENSSL_3.0.0)(64bit) libgcc_s.so.1()(64bit) libgcc_s.so.1(GCC_3.0)(64bit) libm.so.6()(64bit) libm.so.6(GLIBC_2.2.5)(64bit) libm.so.6(GLIBC_2.29)(64bit) libssl.so.3()(64bit) libssl.so.3(OPENSSL_3.0.0)(64bit) libstdc++.so.6()(64bit) libstdc++.so.6(CXXABI_1.3)(64bit) libstdc++.so.6(CXXABI_1.3.5)(64bit) libstdc++.so.6(CXXABI_1.3.8)(64bit) libstdc++.so.6(CXXABI_1.3.9)(64bit) libstdc++.so.6(GLIBCXX_3.4)(64bit) libstdc++.so.6(GLIBCXX_3.4.11)(64bit) libstdc++.so.6(GLIBCXX_3.4.14)(64bit) libstdc++.so.6(GLIBCXX_3.4.18)(64bit) libstdc++.so.6(GLIBCXX_3.4.19)(64bit) libstdc++.so.6(GLIBCXX_3.4.20)(64bit) libstdc++.so.6(GLIBCXX_3.4.21)(64bit) libstdc++.so.6(GLIBCXX_3.4.9)(64bit) libz.so.1()(64bit) libz.so.1(ZLIB_1.2.3.5)(64bit)
Checking for unpackaged file(s): /usr/lib/rpm/check-files /root/rpmbuild/BUILDROOT/aria2c-1.37.0-1.x86_64
Wrote: /root/rpmbuild/RPMS/x86_64/aria2c-1.37.0-1.x86_64.rpm
Executing(%clean): /bin/sh -e /var/tmp/rpm-tmp.2u4TWB
+ umask 022
+ cd /root/rpmbuild/BUILD
+ cd aria2-1.37.0
+ rm -rf /root/rpmbuild/BUILDROOT/aria2c-1.37.0-1.x86_64
+ exit 0
```

7) Test działania:

a) Przejście do folderu z gotowymi pakietami:

```
ac18f033d3e2:~/rpmbuild/SPECS # cd ~/rpmbuild/RPMS/x86_64/
ac18f033d3e2:~/rpmbuild/RPMS/x86_64 # ls
aria2c-1.37.0-1.x86_64.rpm
```

b) Zainstalowanie zbudowanej paczki:

```
ac18f033d3e2:~/rpmbuild/RPMS/x86_64 # rpm -ivh aria2c-1.37.0-1.x86_64.rpm
Preparing...                               ##### [100%]
Updating / installing...
 1:aria2c-1.37.0-1                         ##### [100%]
```

c) Weryfikacja:

```
ac18f033d3e2:~/rpmbuild/RPMS/x86_64 # aria2c -v
aria2 version 1.37.0
Copyright (C) 2006, 2019 Tatsuhiko Tsujikawa

This program is free software; you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation; either version 2 of the License, or
(at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

** Configuration **
Enabled Features: BitTorrent, GZip, HTTPS, Message Digest
Hash Algorithms: sha-1, sha-224, sha-256, sha-384, sha-512, md5, adler32
Libraries: zlib/1.2.13 OpenSSL/3.1.0d
Compiler: gcc 7.5.0
built by x86_64-pc-linux-gnu
on Jun 20 2026 15:26:17
system: Linux 6.18.33.1-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC
```

3. Wnioski:

Z pozoru zawila metoda oparta na plikach .spec okazuje się być niezwykle interesująca i z perspektywy użycia fpm o wiele bardziej wartościowa.

Można powiedzieć, że metoda ta wymusza na nas pewnego rodzaju dyscyplinę, której zastosowanie gwarantuje czystość, przejrzystość i powtarzalność - raz zapisany plik aria2.spec powoduje, że aktualizacja oprogramowania do nowej wersji w przyszłości będzie sprowadzać się jedynie do podmiany archiwum i uruchomienia jednej komendy.

Metoda, której używaliśmy na zajęciach jest przystępna i ma niski próg wejścia, jeśli chodzi o zrozumienie idei działania pakietów, lecz takie pójście na skróty często może nas prowadzić do problemów z zależnościami (jak np. w przypadku konfliktu wersji Rubiego i modułu ffi, na które natrafiliśmy, czyli tzw. dependency hell często pojawiające się przy korzystaniu z zewnętrznych menedżerów pakietów - tak jak właśnie gem dla Rubiego).

To pokazuje też słaby punkt narzędzia fpm w środowiskach produkcyjnych. Jest on zależny od ekosystemu Ruby - używając menedżerów zewnętrznych, często pobieramy surowy kod z internetu, więc jeśli autor biblioteki ffi napisał kod z myślą o najnowszej wersji oprogramowania, a my operujemy na starszej - dochodzi do konfliktu. Natomiast natywny rpmbuild - bezpośrednio pobierany przez menedżer systemowy zypper, zawsze będzie w 100% zgodny z resztą naszego systemu. Narzędzia są napisane głównie w języku C, skompilowane przez twórców systemu i perfekcyjnie dopasowane do głównej biblioteki glibc.

Podsumowując - dobrze jest przetestować sobie działanie obu metod, zrozumieć przebieg, różnice, wady i zalety. Robiąc research i widząc różne przypadki, błędy i konflikty podczas korzystania z metody pierwszej, na dłuższą metę, wybrałbym korzystanie z rpmbuild i działanie na plikach .spec.